

Monitorización de un clúster Spark con Grafana

Julio Garcia Ortiz



1. Plataforma de ejecución	3
1.1. Creación del entorno de ejecución	3
1.2. Exploración inicial	4
1.3. Importación y exploración de dashboards	4
2. Exploración de Grafana	5
2.1. ¿Qué componentes intervienen en el flujo de monitorización que llega hasta Grafana?	5
2.2. ¿Qué métrica se utilizó para monitorizar la ejecución del job Spark y qué comportamiento se observó en la gráfica?	6
2.3. ¿Qué panel del dashboard importado se analizó con más detalle y qué información muestra?	6
2.4. ¿Por qué el tipo de gráfico utilizado en ese panel es adecuado para la métrica que representa?	7
3. Construcción del dashboard	7
3.1. Panel de indicador instantáneo (gauge)	7
3.2. Panel de serie temporal	8
3.3. Mapa de calor	8
3.4. Gráfico de barras	9
3.5. Paneles extra	9
3.6. Dashboard completo	11
4. Exportación del dashboard	11

1. Plataforma de ejecución

1.1. Creación del entorno de ejecución

Descargamos el zip adjuntado por el profesor, extraemos los archivos y ejecutamos el `docker compose up -d` para ejecutar el `.yml` que creará el container en docker desktop.

```
jgarort@MacBook-Air-de-Julio spark-lab-prometheus-grafana % docker compose up -d
[+] up 62/62
✓ Image grafana/grafana:latest          Pulled      25.3s
✓ Image apache/spark:3.5.3              Pulled      23.7s
✓ Image prom/node-exporter:latest       Pulled      11.2s
✓ Image gcr.io/cadvisor/cadvisor:latest Pulled      8.3s
✓ Image busybox:1.36                   Pulled      8.1s
✓ Image prom/prometheus:latest          Pulled      12.7s
✓ Network spark-lab-prometheus-grafana_spark-net Created     0.0s
✓ Network spark-lab-prometheus-grafana_default Created     0.0s
✓ Volume spark-lab-prometheus-grafana_grafana-data Created     0.0s
✓ Volume spark-lab-prometheus-grafana_spark-events Created     0.0s
✓ Volume spark-lab-prometheus-grafana_prom-data Created     0.0s
✓ Container init-spark-events           Started     16.9s
✓ Container prometheus                  Started     16.9s
✓ Container web-server-1                Started     0.5s
✓ Container cadvisor                    Started     0.5s
✓ Container spark-master                 Started     16.7s
✓ Container grafana                     Started     16.7s
✓ Container spark-history                Started     16.8s
✓ Container spark-worker-2              Started     16.8s
✓ Container spark-worker-1              Started     16.8s
jgarort@MacBook-Air-de-Julio spark-lab-prometheus-grafana %
```

Container CPU usage ⓘ

2.99% / 1000% (10 CPUs available)

Container memory usage ⓘ

1.22GB / 7.57GB

Show charts

☒ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last ... ↓	Actions
☐	spark-lab-prometheus	-	-	-	3%	30 minutes ago	
☐	init-spark-events	74b3b6f55ea6	busybox:1.36		0%	30 minutes ago	
☐	prometheus	fe0edd4e8e5	prom/prometheus:late	9090:9090	0%	30 minutes ago	
☐	web-server-1	121f3952de04	prom/node-exporter:la		0%	30 minutes ago	
☐	cadvisor	15aa11f3284d	cadvisor/cadvisor:late	8088:8080	1.9%	30 minutes ago	
☐	spark-master	5b33c2d42fa3	apache/spark:3.5.3	4040:4040	0.1%	30 minutes ago	
☐	grafana	f922fdb7de6d	grafana/grafana:lates	3000:3000	0.71%	30 minutes ago	
☐	spark-worker-1	13d87bf65c85	apache/spark:3.5.3	8081:8081	0.08%	30 minutes ago	
☐	spark-worker-2	9e32a9da3f38	apache/spark:3.5.3	8082:8082	0.09%	30 minutes ago	
☐	spark-history	7c46fb0607c2	apache/spark:3.5.3	18080:18080	0.12%	30 minutes ago	

1.2. Exploración inicial

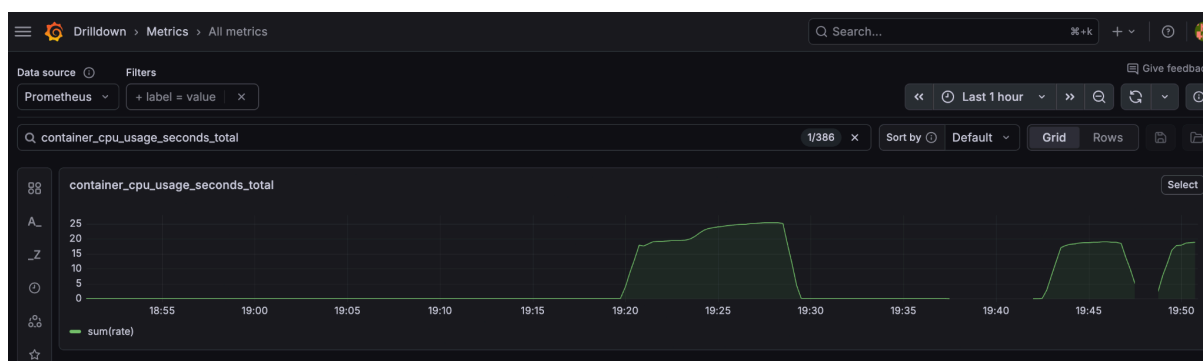
Lanzamos el comando para ejecutar el job.

```
jgarort@MacBook-Air-de-Julio spark-lab-prometheus-grafana % docker exec -it spark-master bash -lc '
/opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
--conf spark.ui.enabled=true \
--conf spark.driver.bindAddress=0.0.0.0 \
--class org.apache.spark.examples.SparkPi \
/opt/spark/examples/jars/spark-examples_2.12-3.5.3.jar 1000000
'
```

Al ejecutar el job y hacer la consulta para ver las métricas encontré el error de que no me estaba cogiendo bien los datos. (captura 1)

`sum(rate(container_cpu_usage_seconds_total{name=~"spark-*"}[10s])) by (name)`

Esa era la consulta exacta a ejecutar pero entrando en prometheus y haciendo esta otra consulta con ayuda del profesor vimos que no está haciendo bien la consulta ya que no busca la etiqueta `name` `name=~"spark-*"` (captura 2)



> container_cpu_usage_seconds_total

Execute

Table Graph Explain

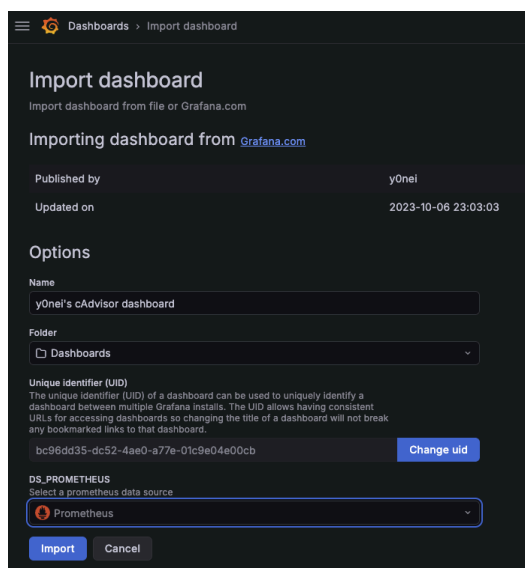
Evaluation time

Load time: 8ms Result series: 11

container_cpu_usage_seconds_total(cpu="total", id="/", instance="cadvisor:8080", job="cadvisor")	1544.62
container_cpu_usage_seconds_total(cpu="total", id="/docker", instance="cadvisor:8080", job="cadvisor")	1473.353698
container_cpu_usage_seconds_total(cpu="total", id="/docker/121f3952de04f319cf11b5b91067d24f9f3069b8521fda268a699ba85231261", instance="cadvisor:8080", job="cadvisor")	1.119642

1.3. Importación y exploración de dashboards

Tal y como nos dicen las instrucciones del profesor importamos el nuevo dashboard con ID 19724 y seleccionamos Prometheus como fuente de datos. No tengo capturas de los pasos anteriores explorando el catálogo de dashboards ya que esa parte la hice en el ordenador de clase y también ayude a un compañero por lo que ya directamente la salte al realizar esta parte de la tarea.



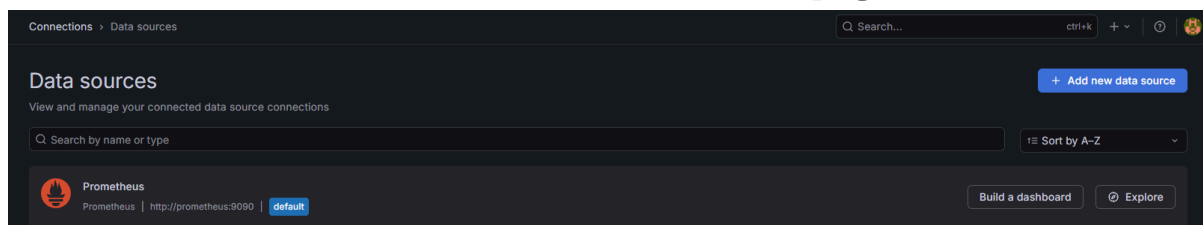
2. Exploración de Grafana

2.1. ¿Qué componentes intervienen en el flujo de monitorización que llega hasta Grafana?

Para empezar ejecutamos el job en spark mediante el cual recopilamos, almacenamos y visualizamos métricas.

El cluster de apache spark ejecuta todos los jobs dentro de los contenedores de docker, estos generan métricas sobre usos de cpu, memoria o actividad del sistema. Estas métricas que se han generadas son recopiladas mediante cadvisor, ésta se encarga de obtener la información que se monitoriza en los contenedores de docker. A continuación prometheus recoge cada x tiempo (cuando se le asigne) las métricas mediante scraping y se almacenan de forma temporal en su base de datos.

Finalmente nos conectamos mediante grafana a prometheus y esto nos permite visualizar los datos obtenidos mediante dashboards con kpi, gráficas etc.



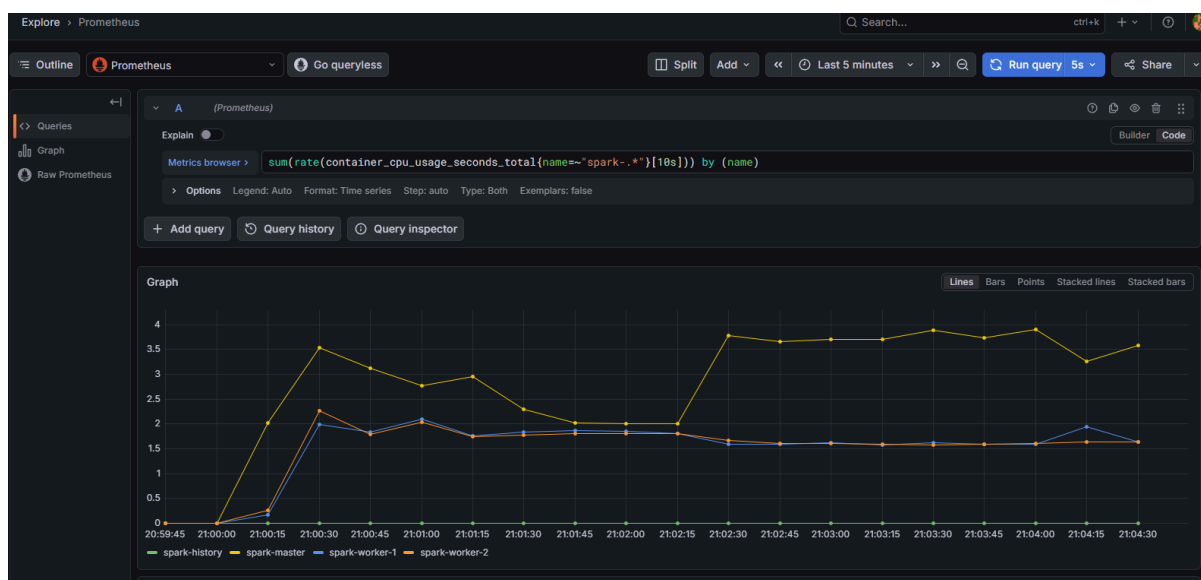
En esta imagen vemos cómo usamos prometheus como fuente de datos.

2.2. ¿Qué métrica se utilizó para monitorizar la ejecución del job Spark y qué comportamiento se observó en la gráfica?

```
sum(rate(container_cpu_usage_seconds_total{name=~"spark-.*"}[10s]) by (name))
```

Con esta consulta podemos ver el tiempo acumulado de uso de la cpu de cada contenedor de docker. Mediante la función `rate()` obtenemos el consumo de cpu en intervalos de 10 segundos y `by(name)` separamos los resultados por contenedores.

Durante la ejecución podemos observar actividad en los diferentes workers y master, en mi caso puse para que la query se auto ejecute cada 5s para ir actualizando automáticamente los datos y podemos observar los picos de consumo.



A diferencia del punto 1.2 ahora si puedo acceder bien a los datos ya que ahora mismo estoy realizando la actividad en otro equipo.

2.3. ¿Qué panel del dashboard importado se analizó con más detalle y qué información muestra?

El panel utilizado fue el de uso de CPU de los contenedores de Spark importado por cadvisor. En este panel se muestra el consumo de cpu de cada contenedor, permitiendo observar la actividad de los distintos nodos como el master o los workers.

La información obtenida nos resulta útil ya que nos permite identificar qué contenedores tienen mayor carga de trabajo y nos permite detectar picos de consumo o posibles errores en el cluster. Gracias a este panel podemos comprobar como aumenta la actividad de CPU mientras corre el job de spark

2.4. ¿Por qué el tipo de gráfico utilizado en ese panel es adecuado para la métrica que representa?

En el panel vemos un gráfico de serie temporal, un tipo de visualización perfecto para representar métricas como las de uso de CPU, ya que nos permite ver cómo evoluciona durante la ejecución. Este gráfico nos permite ver e identificar de forma bastante sencilla los cambios en la actividad de los contenedores y sus picos de consumo.

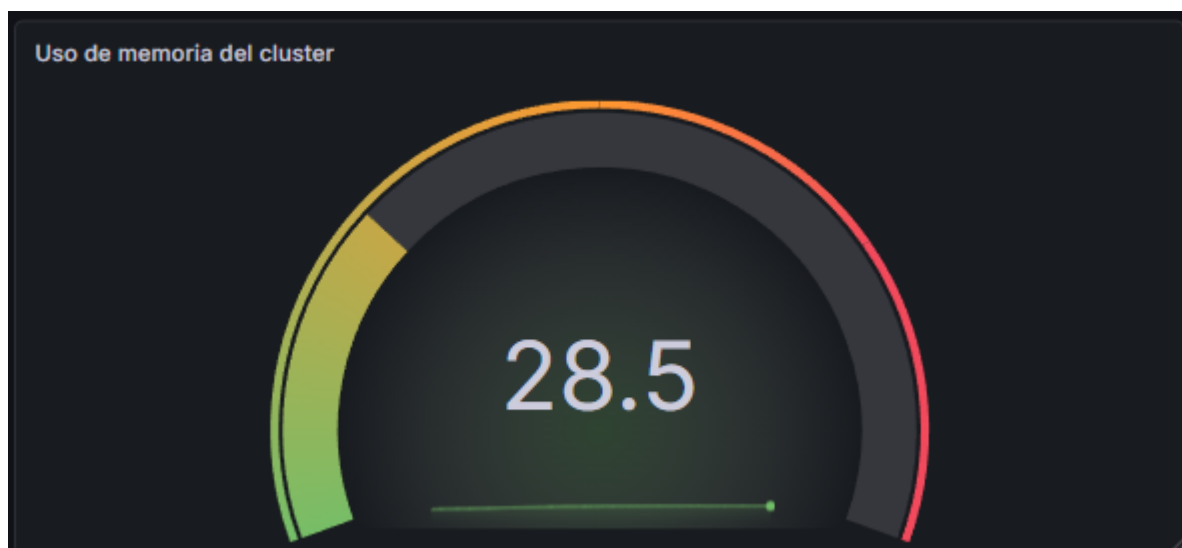
También nos facilita poder diferenciar los distintos contenedores para comparar el comportamiento de los nodos como master, worker1 o worker2 y así poder detectar desequilibrios o problemas de rendimiento.

3. Construcción del dashboard

3.1. Panel de indicador instantáneo (gauge)

En este panel tengo un medidor de gauge donde mostramos el porcentaje de memoria RAM usada por los contenedores del cluster de Spark. Para obtener este dato usamos una consulta que suma la memoria total consumida por los contenedores y la compara con la memoria total de la máquina

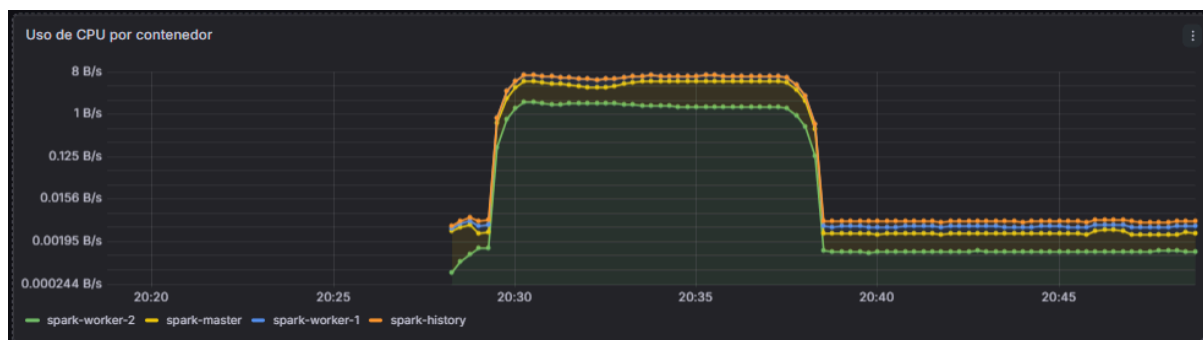
```
sum(container_memory_usage_bytes{name=~"spark-.*"}) / sum(machine_memory_bytes) * 100
```



3.2. Panel de serie temporal

En este podemos observar el tráfico de red entrante por contenedor, utilizamos una serie temporal para ver el uso de CPU de los contenedores a lo largo del tiempo. Con este gráfico podemos identificar fácilmente variaciones o picos de carga, en esta imagen en concreto vemos como estaba en un consumo bajo y estable y cuando ejecuté el job el consumo del master y los trabajadores se disparó y después como volvió a bajar drásticamente cuando finalice el job.

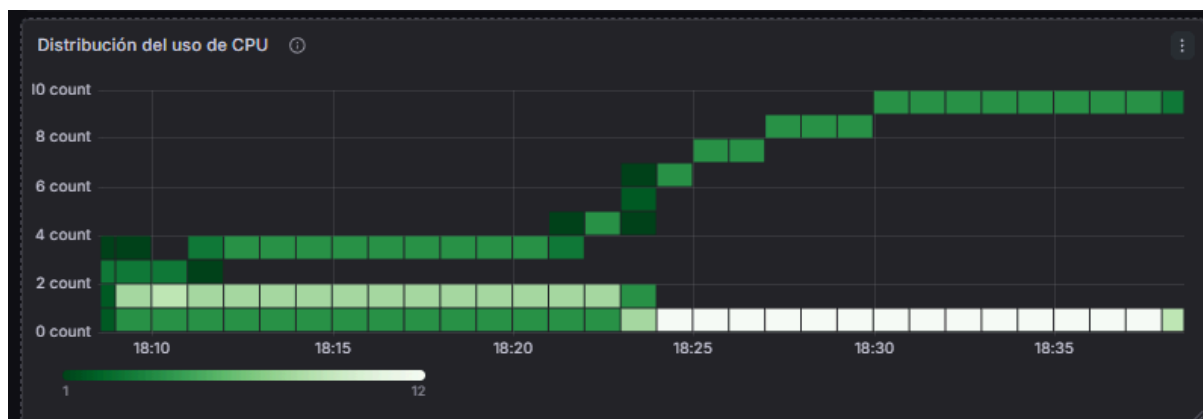
```
rate(container_cpu_usage_seconds_total{name=~"spark-*"}[1m])
```



3.3. Mapa de calor

En este mapa de calor podemos ver la distribución del uso de CPU mediante los contenedores de spark a lo largo del tiempo. Así podemos ver de forma sencilla y rápida las zonas con mayor actividad. Es bastante útil para ver o detectar patrones de carga del cluster.

```
rate(container_cpu_usage_seconds_total{name=~"spark-*"}[1m])
```



3.4. Gráfico de barras

Con este gráfico de barras podemos ver el consumo de memoria individual de cada contenedor. Se agrupan los resultados utilizando `by(name)` para separar la memoria de cada contenedor. De esta forma es bastante fácil de entender y ver el consumo de memoria entre los distintos contenedores ayudándonos a identificar irregularidades o problemas.

```
sum(container_memory_usage_bytes{name=~"spark-*"}) by (name)
```



3.5. Paneles extra

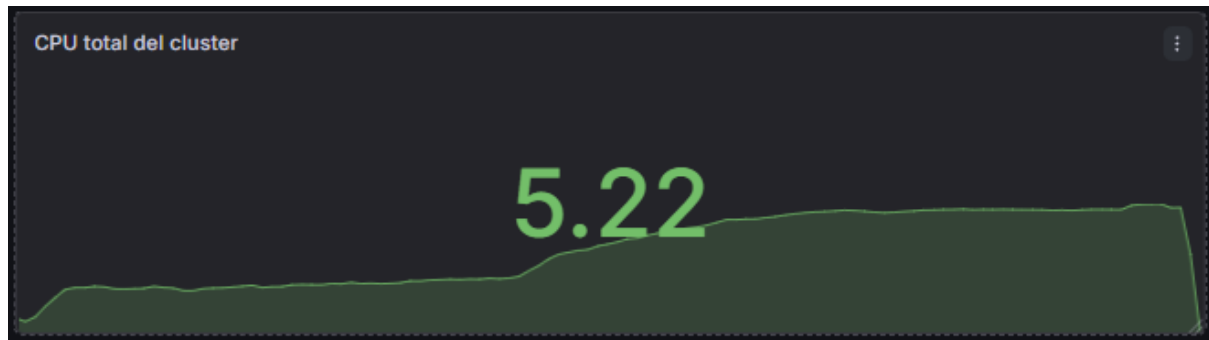
Este panel muestra el número de contenedores Spark activos en el clúster. La consulta cuenta únicamente aquellos contenedores que presentan consumo de memoria superior a cero, permitiendo verificar rápidamente el estado general del entorno distribuido.

```
count(container_memory_usage_bytes{name=~"spark-*"} > 0)
```



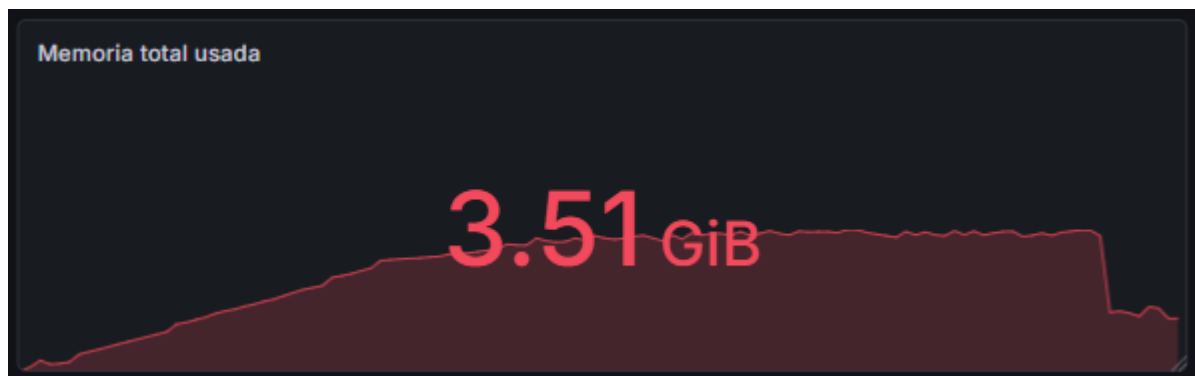
Este panel representa el consumo total de CPU del clúster Spark sumando la actividad de todos los contenedores. Permite visualizar de forma rápida el nivel general de carga de procesamiento durante la ejecución de los jobs.

```
sum(rate(container_cpu_usage_seconds_total{name=~"spark-*"}[1m]))
```



Este panel muestra la cantidad total de memoria utilizada por todos los contenedores Spark del clúster. Resulta útil para monitorizar el consumo global de memoria y detectar posibles situaciones de saturación de recursos.

```
sum(container_memory_usage_bytes{name=~"spark-*"})
```



3.6. Dashboard completo

Esta sería la vista general del dashboard desarrollado en Grafana, donde se pueden observar todos los paneles configurados anteriormente. En conjunto, el dashboard permite monitorizar de forma visual el estado y comportamiento del clúster Spark mediante métricas de CPU, memoria y actividad de los contenedores.



4. Exportación del dashboard

Ya finalmente para terminar la entrega procedemos a exportar el dashboard en formato json y descargamos.

